Alunos: Bruno Hiert Rech e Gabriel Heron Nascimento

Link do projeto:
https://github.com/Bromo367/Sistema-de-consultas-medicas-em-JavaFX

Breve resumo do estado do projeto:

O sistema é uma interface feita em Javafx que permite a manipulação de um banco de dados fictício temático de consultas médicas. O fluxo do projeto atual é feito na seguinte estrutura:
A parte do View é feita em fxml usando o Javafx, os eventos do View são enviados para o Controller que valida as entradas e se necessário chama o DAO da respectiva tabela do banco de dados; O DAO realmente implementa as operações do banco de dados específico, no caso o postgreSQL.

Possíveis falhas na estrutura.

A manutenibilidade do código ainda pode ser melhorada, pois ainda há um acoplamento relativamente alto entre as classes. As classes Controller possuem responsabilidades demais (receber entradas, validá-las e chamar o DAO) ferindo o princípio da responsabilidade única de classes.
O projeto ainda é muito limitado e não apresenta muitas funcionalidades, o usuário simplesmente faz as operações de CRUD nas cinco tabelas do banco e pode também listar todos os registros de cada tabela. Porém, essas tarefas ainda não agregam valor ao software e ele pouco resolveria problemas reais dos usuários.

Camada Service

Pensando nessas falhas estruturais, será implementada uma camada extra na arquitetura do software, a camada Service. Assim, o Controller será responsável apenas por verificar se as entradas podem ser convertidas para os tipos necessários (como inteiros, datas, etc) e criar objetos models que serão enviados à camada Service. Esta por sua vez, será a responsável por verificar as regras de negócio, ou seja não apenas se uma entrada é válida, mas sim se ela faz sentido dentro do contexto específico do programa.

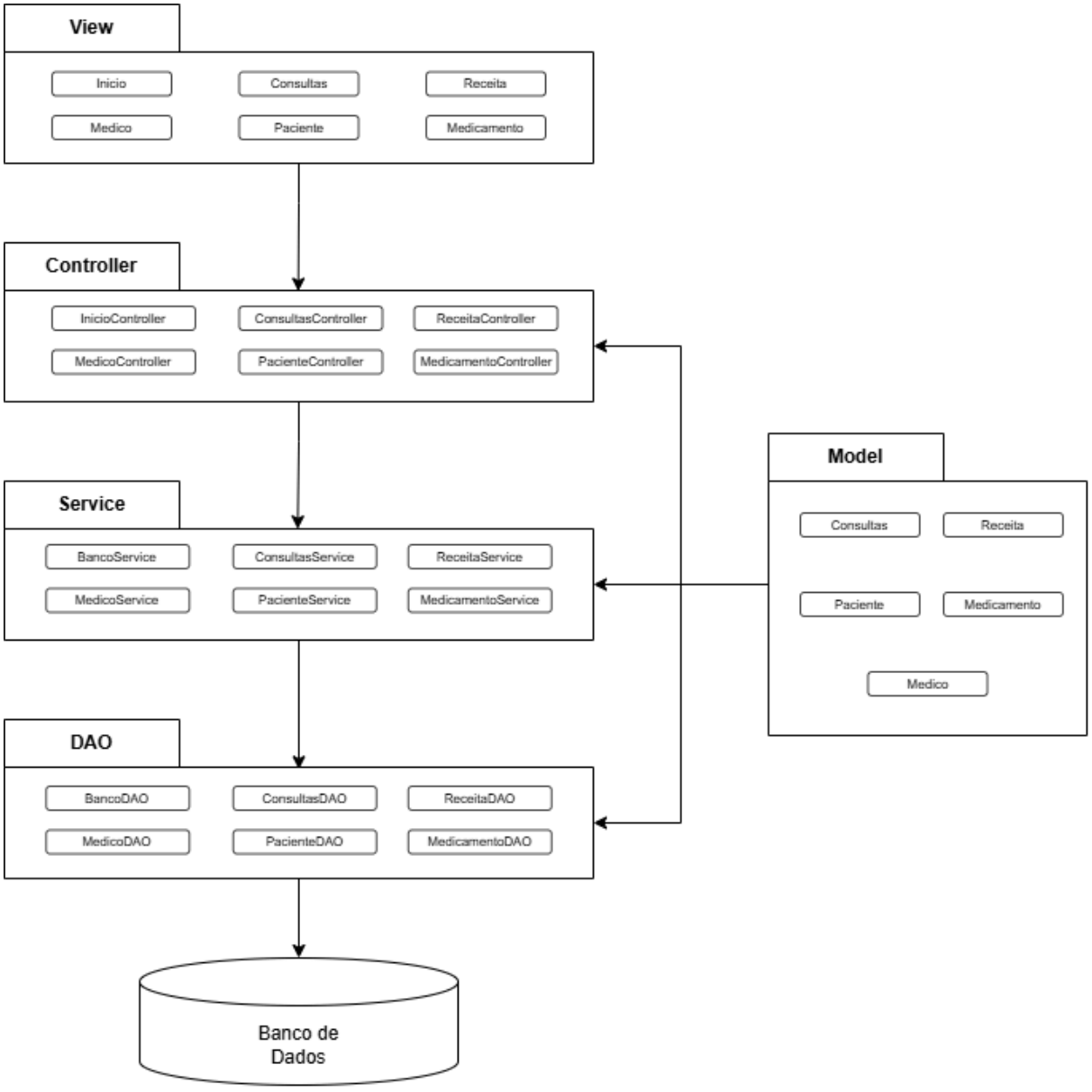
Como benefícios esperados temos uma maior divisão das responsabilidades das classes, um menor acoplamento entre as camadas do sistema e essas mudanças facilitarão manutenção futura do software, fazendo com que alterações grandes como a troca do banco de dados sejam mais facilmente implementadas e impactem o mínimo de classes possível.

Implementação da Solução

Foi criada uma classe service para cada um dos Models do banco, dessa forma cada controller se comunica com o service e este se comunica com o DAO. Assim, o DAO é invisível para o Controller, de modo que se ele for substituído não é necessário nenhuma alteração nos Controllers.

É importante ressaltar, também, que cada classe service não se limita a uma tela específica, assim se fossem implementadas novas telas com funcionalidades iguais a uma já existente, poderia ser utilizada a mesma classe service. Isso também vale para o caso em que fosse feita a portabilidade do sistema para outras telas, como um aplicativo mobile, nesse caso a camada service serviria para ambos os tipos de tela.

Diagrama da Solução



Padrão Builder

A finalidade da implementação do padrão Builder se dá pela necessidade de simplificar e auxiliar na implementação de construtores para as classes. Na execução do projeto, os objetos eram criados dentro das classes Controller, o que fazia com que houvesse uma dificuldade de interpretar, encontrar e fazer a manutenção desses construtores. Com a implementação desse padrão, foi facilitada a visualização dos construtores, sendo divididos de maneira mais adequada.

O padrão foi implementado para as classes ReceitaBuilder, MedicoBuilder e PacienteBuilder e foram necessárias modificações em ReceitaController, MedicoController e PacienteController.

Situação Anterior

O projeto tinha diversos objetos sendo criados de forma simples, passando muitos parâmetros de maneira confusa e desorganizada. Aliado a isso, qualquer alteração de construtor ou de parâmetro estava complicada de encaixar, por conta de conflitos com estruturas previamente construídas.

Situação Implementada

Foi utilizado o padrão de Build para funcionar em 3 entidades principais: Paciente, Médico e Receita. Para todos, foi implementada uma classe Builder respectiva, onde os métodos foram criados, juntamente com o método .build que faz a criação efetiva do objeto, além de modificações da criação dos objetos nas classes Controllers de cada entidade.



```
Receita receita = new ReceitaBuilder()
            .cpfPaciente(cpf)
            .crmMedico(crm)
            .data(data)
            .medicamentoId(id)
            .dose(dose)
            .unidadeMedida(unidade)
            .intervalo(intervalo)
            .build();
```

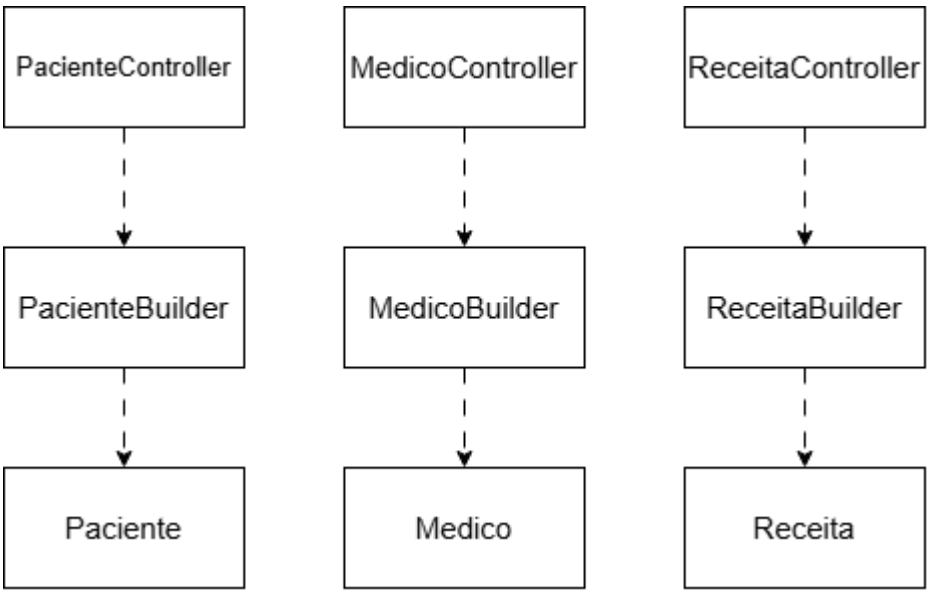


```
public class ReceitaBuilder{
    private String cpfPaciente;
    private String crmMedico;
    private String data;
    private int medicamentoId;
    private double dose;
    private String unidadeMedida;
    private String intervalo;

    public Receita build(){
        return new Receita(cpfPaciente
,crmMedico,data, medicamentoId, dose,
unidadeMedida, intervalo);
    }
    public ReceitaBuilder cpfPaciente(
String cpf){
        this.cpfPaciente = cpf;
        return this;
    }
```

Justificativa e verificação
O benefício esperado é a facilidade de criação de objetos a partir dos Builders e a facilidade de manutenção do código futuramente. A implementação pode ser verificada por meio da utilização das classes Builders nos respectivos Controllers, bem como pela execução das funcionalidades do sistema que realizam a criação dessas entidades.

Diagrama



Padrão Strategy

A implementação do padrão Strategy se dá pela necessidade de uma funcionalidade relacionada à ordenação das listas do método listarTodos(), onde o usuário poderá escolher por qual coluna da tabela ele quer ordenar, garantindo que ele tenha maior poder de customização e organização.

O padrão foi aplicado implementando uma interface de ordenação e outras classes que implementam os métodos de ordenação em si. Houveram classes criadas como OrdenarReceita, OrdenarReceitaCpf, OrdenarReceitaCrm e OrdenarReceitaData, bem como foram feitas modificações em classes como ReceitaService e ReceitaController. Além disso, uma funcionalidade foi adicionada na interface da Receita.

Situação Anterior

Não havia a possibilidade de ordenar a lista de receitas gerada ao utilizar o método listarTodos(), podendo atrapalhar na utilização e visualização das informações.

Situação implementada

Foram implementadas as classes relacionadas aos métodos de ordenação seguindo o padrão de Strategy, sendo a classe OrdenarReceita a interface que as outras classes implementariam. Assim, foram feitos 3 métodos para ordenação: OrdenarReceitaCpf, OrdenarReceitaCrm e OrdenarReceitaData.

```
public interface OrdenarReceita {
    void ordenar(List<Receita> receitas
);
}
```

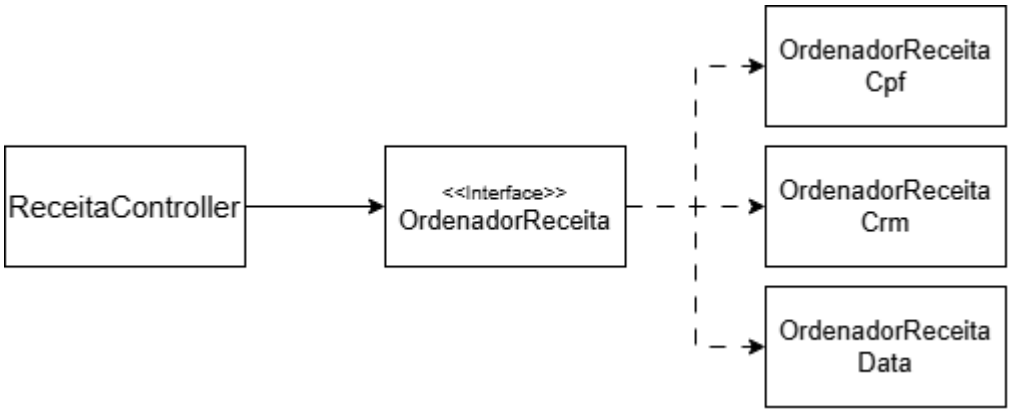
```java
public class OrdenarReceitaCpf
implements OrdenarReceita{

    @Override
    public void ordenar(List<Receita>
receitas){
        receitas.sort((r1, r2) -> r1.
getCpfPaciente().compareTo(r2.
getCpfPaciente()));
    }
}
```

Justificativa e verificação

O benefício esperado é a funcionalidade de ordenação funcionando, garantindo que a lista é criada com o método de ordenação implementado. A implementação pode ser verificada por meio da funcionalidade de listagem de receitas, na qual o usuário pode selecionar uma coluna para realizar a ordenação. Ao selecionar uma opção, a estratégia correspondente é utilizada para ordenar a lista, demonstrando que as diferentes implementações da interface OrdenarReceita estão integradas e funcionando no sistema.

Diagrama



Padrão Observer

A implementação do padrão Observer se dá pela necessidade de atualizar automaticamente a lista de receitas sempre que uma nova receita é adicionada. Antes, era necessário clicar novamente em listarTodos() para visualizar a nova receita. Com o Observer, essa atualização passou a acontecer automaticamente.

Foram criadas as classes ReceitaObserver e ReceitaObserverConcrete, além de alterações feitas nas classes de Service e Controller.

Situação Anterior

Quando uma nova receita era cadastrada, a lista não era atualizada automaticamente, sendo necessário executar novamente o método listarTodos() para visualizar as alterações.

Situação Implementada

Foi implementado o padrão Observer para acompanhar as alterações na lista de receitas. Quando uma nova receita é adicionada, o Observer identifica a atualização e notifica para que a lista seja atualizada automaticamente.

```
import java.util.List;
import br.unicentro.appconsultas.model.
Receita;

public interface ReceitaObserver {
    void listaReceitaAtualizada(List<
Receita> receitas);
}
```

```
@Override
    public void listaReceitaAtualizada
(List<Receita> receitas) {

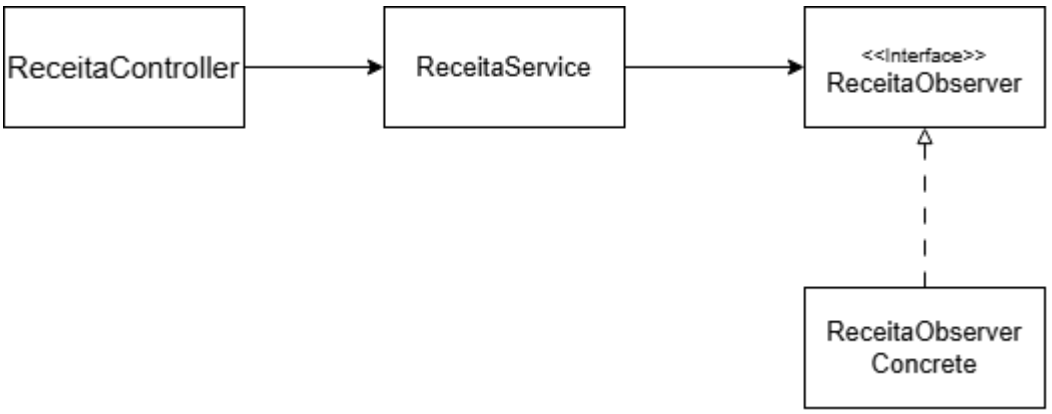
        if(metodo != null){
            metodo.ordenar(receitas);
        }
        tabela.getItems().setAll
(receitas);

        double altura = 28 + (tabela.
getItems().size() * 24) + 5;
        tabela.setPrefHeight(altura);
        tabela.setMinHeight(altura);
        tabela.setMaxHeight(altura);
    }
}
```

Justificativa e Verificação

O benefício esperado é proporcionar maior praticidade ao usuário, evitando a necessidade de atualizar a lista manualmente. A implementação pode ser verificada cadastrando uma nova receita e observando que ela aparece automaticamente na lista, sem precisar clicar novamente em listarTodos().

Diagrama


ReceitaController
ReceitaService
<<Interface>>
ReceitaObserver
ReceitaObserver
Concrete